

Comprehensive Data Science Question Bank Answers: ML & DL

Topic 1: Machine Learning Fundamentals & Data Preprocessing

Level 1: Basics

1. Supervised vs. Unsupervised Learning

- **Supervised Learning:** Trained on labeled data. Example: Predicting house prices based on size (Regression) or classifying emails as spam (Classification).
- **Unsupervised Learning:** Trained on unlabeled data to find patterns. Example: Grouping customers into segments (Clustering).

2. Confusion Matrix Components

- **True Positive (TP):** Model correctly predicted positive.
- **True Negative (TN):** Model correctly predicted negative.
- **False Positive (FP):** Type I error. Model incorrectly predicted positive.
- **False Negative (FN):** Type II error. Model incorrectly predicted negative.

3. One-hot encoding

- It converts categorical variables into binary vectors (1s and 0s). It is necessary because ML algorithms expect numerical inputs. If you assign integers (Red=1, Blue=2), the model assumes an ordinal relationship (Blue > Red), which is incorrect for nominal categories.

4. Handling missing data

- **Deletion:** Remove rows/columns if missing data is minimal.
- **Imputation:** Fill with mean/median/mode (numerical) or most frequent (categorical).
- **Advanced:** Use models like KNN imputation or predict missing values.

Level 2: Intermediate

5. Bias-Variance Tradeoff

- **Bias:** Error from approximating a real-world problem with a simple model. High bias leads to **underfitting**.
- **Variance:** Error from model sensitivity to small fluctuations in training data. High variance leads to **overfitting**.
- **Tradeoff:** Increasing model complexity decreases bias but increases variance. You want the sweet spot minimizing total error.

6. Cross-validation

- It involves splitting data into 'k' subsets. You train on k-1 subsets and evaluate on the remaining one, repeating k times. It's preferred because it provides a more robust, unbiased estimate of performance on unseen data than a single train-test split, which might get a "lucky" or "unlucky" distribution of data.

7. ROC Curve and PR AUC

- **ROC Curve:** Plots True Positive Rate vs False Positive Rate. AUC measures the area under it.
- **PR AUC:** Plots Precision vs Recall. It is preferred when dealing with **highly imbalanced datasets**, because the False Positive Rate in ROC can appear artificially low due to a massive number of True Negatives, making the model look better than it is.

8. L1 (Lasso) vs. L2 (Ridge) Regularization

- **L1 (Lasso):** Adds absolute value of coefficients as penalty. Can shrink coefficients exactly to zero, performing implicit feature selection. Use when you suspect many irrelevant features.
- **L2 (Ridge):** Adds squared magnitude as penalty. Shrinks coefficients evenly but rarely to zero. Use to prevent multicollinearity and when most features are useful.

Level 3: Advanced

9. 99% accuracy on 99% negative dataset

- The model achieves 99% by simply guessing "Negative" for every sample. It learns nothing about the positive class, which is usually the important one.
- **Strategy:** Use Precision, Recall, F1-Score, or PR AUC. Use SMOTE, undersampling, or class weights.

10. L1 Sparsity Math/Geometry

- Geometrically, the L1 penalty is a diamond shape, while L2 is a circle. The loss function's contours are ellipses. The optimal weights are where the loss ellipse touches the regularization boundary.
- Because L1 has sharp corners on the axes, the ellipse is highly likely to intersect exactly on an axis (driving a weight to exactly 0). L2's smooth boundary rarely intersects exactly on an axis.

Level 4: "By Hand" Mathematical Calculations

11. Metrics Calculation

- $TP = 80, FP = 20, FN = 40.$
- **Precision:** $TP / (TP + FP) = 80 / 100 = 0.8$
- **Recall:** $TP / (TP + FN) = 80 / 120 = 0.667$
- **F1-score:** $2 \times (0.8 \times 0.667) / (0.8 + 0.667) = 0.727$

12. Naive Bayes

- $P(A|X) = \frac{P(X|A) \cdot P(A)}{P(X|A) \cdot P(A) + P(X|B) \cdot P(B)}$
- **ParseError: KaTeX parse error: Unexpected end of input in a macro argument, expected '}' at end of input: ...{0.36 (or 36%)}**

Topic 2: Supervised Learning (Regression & Classification)

Level 1: Basics

13. Linear Regression Assumptions

- **Linearity, Independence, Homoscedasticity (constant variance of residuals), Normality of errors.** Violation leads to biased estimates, unreliable confidence intervals, and poor predictions.

Level 2: Intermediate

14. Decision Tree Splits

- Trees split based on the feature that best separates data into distinct classes.

- **Gini Impurity:** Measures the likelihood of incorrect classification. Lower is better.
- **Information Gain:** Measures reduction in entropy (uncertainty) after a split. Higher is better.

15. Random Forest

- It's an ensemble of many decision trees. It uses Bagging (training each tree on a random sample) and random feature selection for splits. It performs better because averaging many trees significantly reduces the high variance (overfitting) of a single deep tree.

16. SVM Kernel Trick

- It allows SVM to operate in a high-dimensional feature space to find non-linear boundaries without explicitly calculating the coordinates of the data in that high-dimensional space, saving massive compute.

17. KNN 'K' choice and Bias-Variance

- **Small K (e.g., 1):** Low bias, high variance (Overfitting - very jagged decision boundary).
- **Large K:** High bias, low variance (Underfitting - smooth, overly generalized boundary).

Level 3: Advanced

18. Gradient Boosting vs Random Forests

- **Random Forest:** Trees are independent and built in parallel. Reduces variance.
- **Gradient Boosting:** Trees are built sequentially. Each new tree fits the *residual errors* of the previous ensemble. Reduces bias (but can overfit if unchecked).

19. SVM Dual Formulation

- The Primal form finds hyperplane weights directly. The **Dual formulation** expresses the problem entirely as *dot products* between pairs of training points. Because it only relies on dot products, we can easily swap the standard dot product with a **Kernel function** to implicitly calculate dot products in infinite-dimensional spaces.

20. Decision Tree same continuous feature split

- **Yes.** A tree might split at Age > 50. In the left branch (Age $\leq$ 50), it could split again at Age > 25. This creates segmented, step-like boundaries along a single continuous dimension.

Level 4: "By Hand" Mathematical Calculations

21. Linear Regression Math

- $\Delta y = 2(\Delta x_1) + 3(\Delta x_2)$
- $\Delta y = 2(2) + 3(-1) = 4 - 3 = 1$. y increases by 1.

22. Gini Impurity & Information Gain

- **Parent:** $1 - ((6/10)^2 + (4/10)^2) = 1 - 0.52 = \mathbf{0.48}$
- **Left:** $1 - ((4/5)^2 + (1/5)^2) = 1 - 0.68 = \mathbf{0.32}$
- **Right:** $1 - ((2/5)^2 + (3/5)^2) = 1 - 0.52 = \mathbf{0.48}$
- **Weighted Gini:** $(0.5 \times 0.32) + (0.5 \times 0.48) = \mathbf{0.40}$
- **Did it improve?** Yes, impurity dropped from 0.48 to 0.40.

Topic 3: Unsupervised Learning

Level 1: Basics

23. K-Means Algorithm

- i. Initialize K centroids. 2. Assign points to closest centroid. 3. Recalculate centroids as means of assigned points. 4. Repeat until convergence.
- **Choosing K:** Use the Elbow Method or Silhouette Score.

Level 2: Intermediate

24. PCA

- PCA is a linear dimensionality reduction technique. It transforms correlated variables into uncorrelated variables (Principal Components). It drops PCs with the lowest variance, reducing dimensions while keeping the maximum possible information (variance) from the original data.

Level 3: Advanced

25. K-Means limitations (Non-spherical clusters)

- K-Means relies on Euclidean distance from a center, which intrinsically assumes clusters are spherical and have similar sizes/densities. It fails on complex shapes (like moons).

- **Solution:** Use density-based clustering like DBSCAN, or Spectral Clustering.

Level 4: "By Hand" Mathematical Calculations

26. K-Means Clustering by Hand

- Points: $\{2, 3, 4, 10, 11, 12, 20, 25, 30\}$. Init: $c_1 = 2, c_2 = 12$.
- **Iter 1 Assignments:** Cluster 1: $\{2, 3, 4\}$. Cluster 2: $\{10, 11, 12, 20, 25, 30\}$
- **Iter 1 Centroids:** $c_1 = 3, c_2 = 18$.
- **Iter 2 Assignments:** Cluster 1: $\{2, 3, 4, 10\}$. Cluster 2: $\{11, 12, 20, 25, 30\}$. (Note: point 10 is dist 7 from c_1 and dist 8 from c_2)
- **Iter 2 Final Centroids:** $c_1 = 19/4 = 4.75, c_2 = 98/5 = 19.6$.

Topic 4: Deep Learning Fundamentals

Level 1: Basics

27. Epoch, Batch, Iteration

- **Epoch:** One full pass of the entire dataset.
- **Batch Size:** Number of samples processed before updating weights.
- **Iteration:** One single weight update (processing one batch).

28. Loss Functions

- Measures the error of predictions. **Regression:** Mean Squared Error (MSE). **Classification:** Cross-Entropy.

29. Activation Functions

- Introduces non-linearity. Without them, a deep network is mathematically equivalent to a single linear layer. Common: ReLU, Sigmoid, Tanh.

Level 2: Intermediate

30. Backpropagation

- The algorithm used to train neural networks. It calculates the gradient of the loss function with respect to each weight using the chain rule, moving backwards from the output to the input.

Weights are updated via optimization (like SGD) using these gradients.

31. Batch vs SGD vs Mini-batch

- **Batch:** Entire dataset for one update (Slow, memory heavy).
- **SGD:** One sample per update (Fast, noisy).
- **Mini-batch:** Small chunks per update (Best balance of speed and stability).

32. Dropout

- Regularization technique. Randomly sets a percentage of neurons to 0 during training to prevent co-adaptation, forcing the network to learn robust, distributed representations and reduce overfitting.

33. Vanishing Gradient & ReLU

- In deep networks with Sigmoid, gradients shrink exponentially as they flow backward, preventing early layers from training. **ReLU** ($y = x$ for $x > 0$) has a derivative of exactly 1 for positive values, meaning gradients don't vanish as they pass through active neurons.

Level 3: Advanced

34. Batch Normalization

- Normalizes layer outputs across the batch, then applies learnable scale and shift parameters. It fixes internal covariate shift, allowing for much higher learning rates and acting as a mild regularizer.

35. Optimization Debugging

- **Oscillating Loss Cause:** Learning rate is too high (overshooting the minimum) or Batch size is too small (too much noise in gradients).
- **Fix:** Lower the learning rate, use a learning rate scheduler, or increase the batch size.

36. Sigmoid Derivative & Vanishing Gradients

- $f'(x) = f(x) \cdot (1 - f(x))$
- The maximum value of this derivative is 0.25 (when $f(x) = 0.5$). When you chain rule through N layers, the gradient is multiplied by numbers ≤ 0.25 repeatedly (e.g., 0.25^N). This causes the gradient to exponentially decay to near 0 for early layers.

Level 4: "By Hand" Mathematical Calculations

37. Backpropagation by Hand

- **Part A (Forward):**

$$z = (0.5 \times 2) + 1 = \mathbf{2}$$

$$a = \text{ReLU}(2) = \mathbf{2}$$

$$y_{pred} = (2 \times 2) - 1 = \mathbf{3}$$

$$L = \frac{1}{2}(3 - 5)^2 = \frac{1}{2}(4) = \mathbf{2}$$

- **Part B (Backward):**

$$\frac{\partial L}{\partial y_{pred}} = (y_{pred} - y_{true}) = 3 - 5 = -2$$

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial y_{pred}} \cdot a = -2 \cdot 2 = \mathbf{-4}$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y_{pred}} \cdot w_2 \cdot \text{ReLU}'(z) \cdot x = -2 \cdot 2 \cdot 1 \cdot 2 = \mathbf{-8}$$

Topic 5: CNNs, RNNs, and Advanced Architectures

Level 1: Basics

38. CNN Pooling

- Reduces spatial dimensions (width/height), which reduces parameters and computation, controls overfitting, and provides translation invariance.

39. RNN concept

- They maintain a "hidden state" (memory) from previous steps to process sequential data.

Drawback: They suffer heavily from the vanishing gradient problem over long sequences.

Level 2: Intermediate

40. Attention Mechanism

- Allows a model to focus on specific, relevant parts of the input sequence dynamically, rather than compressing the whole sequence into a single fixed vector.

41. LSTM cell

- Contains a cell state and three gates (Forget, Input, Output). It specifically solves the vanishing gradient problem of standard RNNs by allowing information to flow unchanged through the cell state highway.

42. Transfer Learning

- Taking a model trained on a massive dataset and fine-tuning it on a smaller, specific task by typically freezing the early feature-extraction layers and retraining the final classification head.

Level 3: Advanced

43. Transformer Scaling Math ($1/\sqrt{d_k}$)

- If the key/query dimension d_k is large, their dot product values grow massive. The softmax function is highly sensitive to large values (it pushes the largest value to 1 and the rest to 0, causing gradients to vanish everywhere). Scaling by $\frac{1}{\sqrt{d_k}}$ normalizes the variance back to 1, keeping softmax in a trainable gradient range.

Level 4: "By Hand" Mathematical Calculations

44. CNN Output Size Math

- Formula: $O = \frac{W-K+2P}{S} + 1$
- $O = (32 - 5 + 0)/1 + 1 = 28$.
- Since there are 10 filters, final dimensions are $28 \times 28 \times 10$.

45. RNN Unrolling By Hand

- $h_t = \max(0, 0.5 \cdot h_{t-1} + 2.0 \cdot x_t - 0.5)$, $h_0 = 0$.
- $x_1 = 1$: $h_1 = \max(0, 0 + 2.0 - 0.5) = 1.5$
- $x_2 = -1$: $h_2 = \max(0, 0.5(1.5) - 2.0 - 0.5) = \max(0, 0.75 - 2.5) = 0$
- $x_3 = 2$: $h_3 = \max(0, 0 + 4.0 - 0.5) = 3.5$

Topic 6: Statistics, Probability & Experimentation

Level 1: Basics

46. Central Limit Theorem

- The CLT states that the sampling distribution of the mean will be normally distributed, as long as the sample size is large enough (usually $n > 30$), regardless of the shape of the original population distribution.
- **Importance:** It allows us to use normal probability distributions to make inferences about population parameters, enabling hypothesis testing and confidence intervals even when data isn't normally distributed.

47. Type I vs. Type II Errors

- **Type I (False Positive):** Rejecting the null hypothesis when it is actually true (e.g., convicting an innocent person, or diagnosing a healthy person with a disease).
- **Type II (False Negative):** Failing to reject the null hypothesis when it is false (e.g., letting a guilty person go, or missing a cancer diagnosis).
- **When Type II is worse:** In medical screening (like cancer detection), a False Negative (missing the disease) can be fatal, whereas a False Positive just leads to more testing.

Level 2: Intermediate

48. p-value

- The p-value is the probability of obtaining test results at least as extreme as the results actually observed, under the assumption that the null hypothesis is correct.
- **Usage:** If the p-value is less than a chosen significance level (like 0.05), we reject the null hypothesis, concluding the result is statistically significant.

49. A/B Testing Setup & Sample Size

- **Setup:** Define hypothesis, randomly assign users to Control (A) and Variant (B), run the test, measure metric, perform statistical test (like t-test or Z-test).
- **Sample Size:** Determined *before* the test using Power Analysis. It depends on: Baseline conversion rate, Minimum Detectable Effect (MDE), Statistical Power (usually 80%), and Significance Level (usually 5%).

Level 3: Advanced

50. Novelty & Network Effects in A/B Testing

- **Novelty Effect:** Users interact with a new feature simply because it's new, causing an initial spike in engagement that drops off over time. **Fix:** Run the test longer to let the effect wear off.

- **Network Effect:** Occurs when the behavior of users in the control group is affected by users in the variant group (e.g., in a social network or two-sided marketplace). This violates the independence assumption (SUTVA). **Fix:** Use cluster randomization (e.g., by city instead of by user).

Level 4: "By Hand" Mathematical Calculations

51. Binomial Distribution

- Formula: $P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$
- $n = 5$ (flips), $k = 3$ (heads), $p = 0.6$.
- $\binom{5}{3} = \frac{5!}{3!(5-3)!} = \frac{120}{6 \times 2} = 10$
- $P(X = 3) = 10 \times (0.6)^3 \times (0.4)^2$
- **ParseError: KaTeX parse error: Unexpected end of input in a macro argument, expected '}' at end of input: ...56 (or 34.56%)}**

Topic 7: Advanced Data Processing & Evaluation

Level 1: Basics

52. Normalization vs. Standardization

- **Normalization (Min-Max):** Scales data to a fixed range, usually $[0, 1]$. Use when you need bounded values (e.g., image pixels) or for algorithms not assuming normality (like KNN or Neural Networks).
- **Standardization (Z-score):** Centers data around a mean of 0 with a standard deviation of 1. Preserves outliers. Use when algorithms assume Gaussian distributions (like Linear Regression, SVMs, or PCA).

53. Regression Metrics

- **MSE (Mean Squared Error):** Average squared differences. Punishes large errors heavily.
- **RMSE (Root MSE):** Square root of MSE. Interpretable in the same units as the target variable.
- **MAE (Mean Absolute Error):** Average of absolute differences. Robust to outliers.
- **R-squared:** Proportion of variance in the dependent variable explained by the model (1 is perfect, 0 means acting like a simple mean prediction).

Level 2: Intermediate

54. Outlier Detection & Handling

- **Detection:**
 - **Z-score:** Values > 3 or < -3 std devs from the mean (assumes normal distribution).
 - **IQR:** Values outside $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$. Robust to non-normal distributions.
- **Handling:** Remove them (if they are errors), Cap/Clip them (Winsorization), Transform them (Log transform), or use robust models (Tree-based models are insensitive to outliers).

55. Curse of Dimensionality

- As the number of features (dimensions) grows, the volume of the space increases exponentially, making data points extremely sparse.
- **Effect on KNN:** In high dimensions, the distance between the "nearest" neighbor and the "farthest" neighbor becomes almost identical, making distance metrics useless and causing KNN to fail.

Level 3: Advanced

56. Grid Search vs. Random Search

- **Grid Search:** Exhaustively tests every possible combination of hyperparameters in a defined grid. Slow and computationally expensive.
- **Random Search:** Randomly samples combinations from a statistical distribution.
- **Efficiency:** Random search is often more efficient because not all hyperparameters are equally important. Grid search wastes time testing every value of an unimportant parameter, whereas random search explores a wider variety of values for the *important* parameters in the same amount of time.

Level 4: "By Hand" Mathematical Calculations

57. Z-Score Calculation

- Formula: $Z = \frac{x - \mu}{\sigma}$
- $x = 9, \mu = 5, \sigma = 2$.
- $Z = \frac{9 - 5}{2} = \frac{4}{2} = 2.0$
- Since $2.0 < 3$, it is **not** an outlier under a standard threshold of 3.

Topic 8: NLP, Generative AI & LLMs

Level 1: Basics

58. BoW vs. TF-IDF

- **BoW (Bag of Words):** Simply counts the frequency of each word in a document. Ignores context and grammar.
- **TF-IDF:** Multiplies word frequency by its inverse document frequency. It lowers the weight of very common words (like "the") and increases the weight of rare, informative words.

59. Word Embeddings

- Dense, low-dimensional continuous vectors representing words. (e.g., 300 dimensions instead of a 50,000 dimension one-hot vector).
- **Superiority:** They capture semantic meaning and relationships (e.g., $\text{King} - \text{Man} + \text{Woman} \approx \text{Queen}$). Words with similar meanings are close together in the vector space, whereas one-hot vectors are all orthogonal and equidistant.

Level 2: Intermediate

60. Encoder vs. Decoder Architectures

- **Encoder-only (BERT):** Uses bidirectional self-attention to understand context from both left and right simultaneously. Excellent for classification, NER, and understanding text.
- **Decoder-only (GPT):** Uses masked (unidirectional) self-attention, meaning it can only look at past words. Excellent for generating text one token at a time.

61. RAG (Retrieval-Augmented Generation)

- A technique to give LLMs access to external, up-to-date, or private knowledge without retraining them.
- **Pipeline:** 1. Chunk documents and convert to embeddings. 2. Store in a Vector DB. 3. When a user asks a question, embed the query. 4. Perform a similarity search in the DB to retrieve relevant chunks. 5. Pass the chunks + query to the LLM to generate a grounded answer.

Level 3: Advanced

62. PEFT & LoRA

- **PEFT:** Fine-tuning only a small subset of parameters instead of the entire massive model.
- **LoRA:** Instead of updating the massive weight matrices of the LLM directly, LoRA freezes them and injects two small, low-rank matrices (A and B) into the layers. The update is represented as $W_{new} = W + A \times B$. This reduces the number of trainable parameters by 10,000x, allowing massive models to be tuned on a single GPU.

Level 4: "By Hand" Mathematical Calculations

63. TF-IDF Calculation

- Doc 1: "the cat sat" (Total words = 3)
- Doc 2: "the dog sat on the mat" (Total words = 6)
- **TF for "cat" in Doc 1:** Frequency of "cat" / Total words = $1/3$.
- **IDF for "the":** Total Docs / Docs containing "the". Total Docs = 2. Docs with "the" = 2.
Formula: $\log(\frac{N}{df}) = \log(\frac{2}{2}) = \log(1) = 0$.
(Notice how TF-IDF drives the score of the completely uninformative word "the" to 0).

Topic 9: MLOps, System Design & Interpretability

Level 1: Basics

64. Data Drift vs. Concept Drift

- **Data Drift:** The distribution of the input features (X) changes over time (e.g., users get older, sensors degrade).
- **Concept Drift:** The relationship between the features and the target (y) changes over time (e.g., what constituted a "fraudulent" transaction in 2010 is different than in 2024).

Level 2: Intermediate

65. Explainable AI (SHAP / LIME)

- **SHAP:** Based on game theory. It calculates the marginal contribution of each feature to the model's output across all possible coalitions of features. It gives a consistent, mathematically sound feature importance for every single prediction.
- **LIME:** Perturbs the input data, sees how the complex model's predictions change, and trains a simple, interpretable linear model locally around that specific prediction to explain it.

Level 3: Advanced

66. Cold Start Problem in Recommendations

- Occurs when a new user joins and has no interaction history, so Collaborative Filtering fails.
- Solutions:** 1. Content-based filtering (ask user for preferences during onboarding). 2. Recommend global popularity trends. 3. Use demographic data to find similar user segments. 4. Multi-armed bandits to explore and quickly learn preferences.

Level 4: "By Hand" Mathematical Calculations

67. Data Drift (PSI) Calculation

- Formula: $PSI = \sum (Actual\% - Expected\%) \times \ln\left(\frac{Actual\%}{Expected\%}\right)$
- Expected (Train): Bin A = 0.4, Bin B = 0.6
- Actual (Prod): Bin A = 0.5, Bin B = 0.5
- Bin A PSI:** $(0.5 - 0.4) \times \ln(0.5/0.4) = 0.1 \times \ln(1.25) \approx 0.1 \times 0.223 = \mathbf{0.0223}$
- Bin B PSI:** $(0.5 - 0.6) \times \ln(0.5/0.6) = -0.1 \times \ln(0.833) \approx -0.1 \times -0.182 = \mathbf{0.0182}$
- Total PSI:** $0.0223 + 0.0182 = \mathbf{0.0405}$. (A PSI < 0.1 generally indicates no significant drift).

Topic 10: Recommendation Systems & Time Series

Level 1: Basics

68. Collaborative vs. Content-Based Filtering

- Collaborative Filtering:** Recommends items based on the preferences of similar users (User-User) or similar items based on user interaction histories (Item-Item). "Users who liked this also liked..."
- Content-Based Filtering:** Recommends items based on the features/metadata of the items themselves (e.g., genre, author, description) compared to the user's past liked items.

69. Stationarity in Time Series

- A time series is stationary if its statistical properties (mean, variance, autocorrelation) are constant over time.
- Importance:** Most time series models (like ARIMA) mathematically assume stationarity because they rely on historical patterns remaining consistent into the future.

Level 2: Intermediate

70. Matrix Factorization

- It decomposes a massive, sparse User-Item interaction matrix into two smaller, dense matrices: a User-Latent-Feature matrix and an Item-Latent-Feature matrix. The dot product of a user's vector and an item's vector predicts the missing rating, effectively discovering hidden features (like "affinity for action movies") automatically.

71. ARIMA Components

- **AR (Auto-Regressive):** Uses past values in the time series to predict future values.
- **I (Integrated):** Differencing the raw observations to make the time series stationary (subtracting the previous step from the current step).
- **MA (Moving Average):** Uses past forecast errors in a regression-like model to predict future values.

Level 3: Advanced

72. Ranking Metrics (NDCG & MAP@K)

- **NDCG:** Evaluates ranking quality by assigning a highly weighted score to relevant items placed at the very top of the list, and heavily discounting relevant items placed further down. It is normalized against the "perfect" ideal ranking.
- **MAP@K:** Calculates the mean of the Average Precision across all users, looking only at the top K recommendations. It handles binary relevance (relevant or not).

73. Handling Seasonality and Trend

- **Trend:** Remove by differencing (subtracting y_{t-1} from y_t) or fitting a linear regression line and subtracting it.
- **Seasonality:** Remove by seasonal differencing (e.g., subtracting the value from exactly one year ago) or using decomposition techniques (STL decomposition) to separate the signal into Trend, Seasonality, and Residuals.

Level 4: "By Hand" Mathematical Calculations

74. NDCG Calculation

- Formula: $DCG = \sum \frac{rel_i}{\log_2(i+1)}$ for index $i = 1, 2, 3...$
- **Actual DCG:** $[3, 0, 2]$

- $i = 1: 3 / \log_2(2) = 3/1 = 3$
- $i = 2: 0 / \log_2(3) = 0$
- $i = 3: 2 / \log_2(4) = 2/2 = 1$
- $ActualDCG = 3 + 0 + 1 = 4$
- **Ideal DCG (IDCG):** Sorted highest first: $[3, 2, 0]$
 - $i = 1: 3 / \log_2(2) = 3$
 - $i = 2: 2 / \log_2(3) \approx 2/1.585 \approx 1.26$
 - $i = 3: 0$
 - $IDCG = 3 + 1.26 + 0 = 4.26$
- **NDCG:** ParseError: KaTeX parse error: Unexpected end of input in a macro argument, expected '}' at end of input: ...939 (or 93.9%)}

Topic 11: SQL, Data Wrangling & OA Preparation

Level 1: Basics

75. SQL Joins

- **INNER JOIN:** Returns only records that have matching values in both tables.
- **LEFT JOIN:** Returns all records from the left table, and matched records from the right (fills with NULLs if no match).
- **RIGHT JOIN:** Opposite of LEFT JOIN.
- **FULL OUTER JOIN:** Returns all records when there is a match in either left or right table.

76. Pandas `loc` vs `iloc`

- `loc` gets rows/columns with particular **labels** (the index string/name).
- `iloc` gets rows/columns at particular **integer positions** (the actual row number 0, 1, 2...).

Level 2: Intermediate

77. SQL Window Functions (Ranking)

- **ROW_NUMBER():** Gives a unique sequential integer to every row (1, 2, 3, 4) even if there are ties.
- **RANK():** Gives the same rank to ties, but skips the next numbers (e.g., 1, 2, 2, 4).
- **DENSE_RANK():** Gives the same rank to ties, but does *not* skip numbers (e.g., 1, 2, 2, 3).

78. Vectorization in Pandas/NumPy

- Vectorization applies operations to entire arrays simultaneously using highly optimized C-code under the hood, rather than iterating through elements one by one in Python (which has massive overhead). It can be 100x to 1000x faster than `.apply()` or `for` loops.

Level 3: Advanced

79. Optimizing Slow SQL Queries

- i. Analyze the **Execution Plan** (`EXPLAIN`) to see where time is spent (e.g., Full Table Scans).
- ii. Create **Indexes** on columns heavily used in `WHERE` , `JOIN` , or `GROUP BY` clauses.
- iii. Avoid `SELECT *` ; only pull needed columns.
- iv. Replace subqueries with `JOIN` s or CTEs if optimized better by the engine.
- v. Implement table **Partitioning** (e.g., by date) to restrict scans to relevant data chunks.

Level 4: "By Hand" Mathematical Calculations

80. SQL Execution by Hand

- **Data:** (1, 100), (1, 200), (2, 50), (2, 50), (3, 300)
- **GROUP BY & SUM:**
 - Employee 1: $100 + 200 = 300$
 - Employee 2: $50 + 50 = 100$
 - Employee 3: 300
- **HAVING > 100:**
 - Employee 1 (300) -> Keep
 - Employee 2 (100) -> Filter out (must be *greater* than 100)
 - Employee 3 (300) -> Keep
- **ORDER BY total_sales DESC:**
 - Both are 300, so they tie (order depends on engine implementation, usually original order or index).
- **Exact Output:**

employee_id	total_sales
1	300
3	300

Topic 12: Core Mathematical Foundations

Level 1: Basics

81. Dot Product

- Algebraically, it is the sum of the products of corresponding entries of two sequences of numbers.
- Geometrically, if the two vectors are normalized (length of 1), their dot product is exactly the **cosine of the angle between them**, representing their similarity (1 = identical direction, 0 = orthogonal/unrelated, -1 = opposite).

Level 2: Intermediate

82. Eigenvectors and Eigenvalues in PCA

- An **eigenvector** is a non-zero vector that changes at most by a scalar factor (its **eigenvalue**) when a linear transformation is applied to it.
- In **PCA**, we calculate the covariance matrix of the data and find its eigenvectors and eigenvalues. The eigenvectors represent the *directions* of maximum variance (the Principal Components), and the eigenvalues represent the *magnitude* of variance explained by that component.

Level 3: Advanced

83. MLE vs MAP and Regularization

- MLE (Maximum Likelihood Estimation)**: Finds the parameters that maximize the probability of observing the training data. (Assumes no prior knowledge).
- MAP (Maximum A Posteriori)**: Finds the parameters that maximize the probability of observing the data *given a prior belief* about the parameters (using Bayes' theorem).
- Relation to Regularization**: Using MAP with a Gaussian prior (believing weights should be near zero) is mathematically equivalent to L2 Regularization (Ridge). Using MAP with a Laplace prior is equivalent to L1 Regularization (Lasso).

Level 4: "By Hand" Mathematical Calculations

84. Matrix Multiplication

- $$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, x = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

- Result vector y will be a 2×1 matrix.
- $y_1 = (1 \times 5) + (2 \times 6) = 5 + 12 = \mathbf{17}$
- $y_2 = (3 \times 5) + (4 \times 6) = 15 + 24 = \mathbf{39}$
- Output: $\begin{bmatrix} 17 \\ 39 \end{bmatrix}$

Topic 13: Business Acumen, Product Sense & Strategy

Level 1: Basics

85. Product Metrics (DAU/MAU, CAC, LTV)

- **DAU/MAU Ratio:** Daily Active Users divided by Monthly Active Users. It measures user engagement and "stickiness" (e.g., a 50% ratio means the average user opens the app 15 days a month).
- **CAC (Customer Acquisition Cost):** Total marketing/sales cost to acquire one new user.
- **LTV (Customer Lifetime Value):** The total revenue a business expects from a single customer account throughout their relationship.
- **Importance:** A Data Scientist must ensure models drive $LTV > CAC$. If an ML model increases engagement (DAU/MAU) but ruins LTV, it's a failure.

86. Offline vs. Online Metrics

- **Offline Metrics:** Calculated on historical training/validation data (e.g., F1-score, RMSE, AUC). They evaluate if the model learned the math.
- **Online Metrics:** Calculated in live production via A/B testing (e.g., Click-Through Rate, Conversion Rate, Revenue per User). They evaluate if the model actually drives business value. A model can have a great offline AUC but fail to improve online CTR.

Level 2: Intermediate

87. Translating Business to ML (Churn Prediction)

- **1. Define the Target (Label):** What exact action defines "cancellation"? (e.g., user hits the cancel button, or user's payment fails and 30 days pass?).
- **2. Define the Timeframe:** You must predict churn *before* it happens so the business can intervene. E.g., "Given data from Month 1 and 2, predict if the user will churn in Month 3."

- **3. Define the Features:** Engagement metrics (logins, session length), customer support tickets, payment history, etc., all calculated *prior* to the prediction window to prevent data leakage.

88. Simple Interpretable vs. Complex Black-Box Models

- **Use Simple (Interpretable):** When the cost of a wrong decision is massive and requires legal/human justification (e.g., medical diagnosis, denying a bank loan, regulatory compliance). Also, when data is small or latency requirements are extremely strict.
- **Use Complex (Black-Box):** When maximum accuracy translates directly to massive revenue and interpretability doesn't matter to the end-user (e.g., ad targeting, movie recommendations, image recognition).

Level 3: Advanced

89. Investigating a 5% Retention Drop

- **1. Verify the Data:** Is the drop real, or is there a data pipeline bug/logging error?
- **2. Segment the Drop:** Did it drop globally, or only for specific segments? (e.g., only iOS users, only users in India, only newly registered users).
- **3. Check External Factors:** Was there a holiday? Did a competitor launch a feature? Was there a major news event?
- **4. Check Internal Factors:** Did we release a new app update last week? Did an ML model get retrained and pushed to production? Did the server experience downtime?

90. Accuracy vs. Compute/Latency Tradeoff

- A 1% accuracy bump isn't automatically worth it. I would run an **A/B test** to measure the actual *online* business impact of that 1% (e.g., does it actually increase user purchases?).
- Then, calculate the **ROI (Return on Investment)**: If the 1% offline bump translates to 10,000 *extrarevenuepermonthonline*, but the 20015,000 per month, **do not deploy**.
- Regarding latency: Check if adding 50ms degrades the user experience. If it causes users to abandon the page, the offline accuracy gain is worthless.

Level 4: "By Hand" Mathematical Calculations

91. Cost-Benefit Matrix Analysis

- **Costs:** $FP = 10, FN = 500, TP = 0, TN = 0$.
- **Confusion Matrix:** $TP = 10, FP = 20, FN = 5, TN = 65$.
- **Calculate ML Model Cost:**

- Cost from False Positives: $20 \times \$10 = \200
- Cost from False Negatives: $5 \times \$500 = \$2,500$
- Total ML Cost = $\$200 + \$2,500 = \mathbf{\$2,700}$
- **Decision:** The ML model costs 2,700 in errors. The dumb rule – based system costs 3,000. **Yes, you should deploy the ML model**, as it saves the business \$300 per 100 transactions compared to the current baseline, despite making some expensive False Negative errors.